

Implementación computacional y arquitectura del sistema de ruteo de vehículos

El presente documento describe la implementación computacional del modelo de ruteo propuesto, abarcando el preprocesamiento de datos, la construcción de la instancia, la resolución del problema y la visualización de resultados. Su propósito es garantizar la reproducibilidad del estudio, la trazabilidad de las decisiones de modelado y la posibilidad de extender el modelo a nuevos escenarios operativos.

A diferencia del documento de modelación matemática, este apartado no formula el problema, sino que documenta su materialización computacional, describiendo la arquitectura del sistema y las decisiones técnicas adoptadas durante su desarrollo.

Arquitectura General - pipeline

- 1-Preprocesado.py – Obtención de coordenadas de los puntos de interés.
- 2-Distancias.py – Obtención de rutas y matrices reales mediante ORS.
- 3-ProblemData.py – Construcción de la instancia VRP con splitting, tiempos y restricciones.
- 4-Solve.py – Resolución del VRP, impresión de resultados y visualización en Folium.

Repositorio completo disponible en github.com/auramolina/VRP

Entorno computacional

El entorno computacional define las herramientas de desarrollo, el lenguaje de programación y las dependencias externas requeridas para la implementación y ejecución del modelo de ruteo. Su especificación es fundamental para garantizar la reproducibilidad de los resultados y la correcta integración de los distintos componentes del sistema.

Requerimientos software

Para el desarrollo e implementación del modelo se empleó el siguiente entorno computacional:

- Visual Studio Code, como entorno de desarrollo para la edición y ejecución de los scripts.
- Python 3.13, como lenguaje de programación principal.
- Cuenta y clave API de OpenRouteService, necesarias para el acceso a los servicios de cálculo de rutas, distancias y tiempos de viaje. Ver <https://account.heigit.org/>

Librerías

- **OpenRouteService (ORS):** Cálculo de distancias y tiempos de viaje sobre la red vial real, considerando perfil vehicular y restricciones espaciales.
- **Folium:** Visualización geoespacial interactiva de puntos y rutas resultantes del modelo.
- **PyVRP:** Modelado y resolución del problema de ruteo de vehículos con múltiples restricciones (Wouda et al, 2024).
- **Pandas:** Lectura y manipulación de datos tabulares desde archivos CSV.
- **Numpy:** Manejo eficiente de matrices numéricas de distancias y tiempos.
- **Os:** Gestión de archivos y directorios del sistema.
- **Re:** Procesamiento de cadenas mediante expresiones regulares para normalizar nombres de nodos.
- **Json:** Lectura y escritura de archivos GeoJSON con información de rutas.
- **Pickle:** Serialización y almacenamiento de modelos y resultados computacionales.

Recomendaciones

Se recomienda trabajar localmente con el proyecto para evitar problemas de permisos, sincronización o dependencias. Para ello, se sugiere descargar o clonar el repositorio (git clone <https://github.com/auramolina/VRP.git>)

De manera opcional, especialmente si se utilizan múltiples proyectos de Python, o se trabaja habitualmente en Visual Studio Code, se recomienda crear un entorno virtual aislado (`python -m venv .venv`), y activarlo (`.venv\Scripts\activate`) antes de cualquier archivo.

Finalmente, se deben instalar las dependencias necesarias para la correcta ejecución del modelo, las cuales se encuentran especificadas en el archivo `requirements.txt` (`pip install -r requirements.txt`)

Funciones auxiliares: funciones.py

Con el fin de mejorar la legibilidad, reutilización y mantenimiento del código, se implementó un módulo independiente denominado `funciones.py` (ver <https://github.com/auramolina/VRP/blob/main/funciones.py>), el cual agrupa funciones auxiliares desarrolladas progresivamente a lo largo del proceso de implementación.

- **Preprocesamiento espacial:** Preparación de datos geográficos para compatibilidad con ORS y PyVRP.
 - `procesar_wkt_csv`
 - `make_avoid_multipolygon`
- **Gestión de rutas y matrices:** Reutilización de rutas previamente calculadas y desacoplamiento del modelo respecto a ORS.
 - `cargar_geojson`
 - `matrices`
- **Normalización de nombres:** Resolver la divergencia entre clientes reales y subnodos generados en la modelación de la instancia.
 - `clean_name`

- **Estrategias de modelado de demanda:** Implementación de una heurística de Split delivery.
 - Split_FF2S
- **Postprocesamiento de soluciones:** Reconstrucción lógica de la solución para análisis operativo y visualización.
 - agrupar_eventos

Procesamiento de coordenadas: 1-Preprocesado.py

El primer paso realizado fue identificar los puntos de interés, y obtener sus coordenadas, en este caso, las plantas de oriente y, el centro de integración y de distribución. (Ver https://www.google.com/maps/d/u/0/edit?mid=1cgAPynNC_zae6XlQymb6DJDkCcNQM&usp=sharing)

La Secretaría de Movilidad de Medellín restringe la circulación de vehículos con carga superior a 4 toneladas por ciertas vías urbanas, entre las que se incluyen la vía Las Palmas y el Alto de Santa Elena (Para más información, ver <https://www.medellin.gov.co/es/secretaria-de-movilidad/transporte-carga/>). En consecuencia, se identificaron y georreferenciaron los puntos correspondientes a estas zonas con el fin de incorporarlos como restricciones espaciales en el cálculo de rutas. Adicionalmente, se seleccionaron otros puntos estratégicos de exclusión que permiten inducir el direccionamiento hacia la Ruta Nacional 60, asegurando que las rutas calculadas sigan los corredores viales adecuados para vehículos de carga pesada.

Entradas: "Oriente.csv" y "Oriente- avoid.csv"

Archivos .csv en formato:

```
Oriente.csv
1 WKT,Nombre,Descripción
2 "POINT (-75.57969539999999 6.2171745)",CI,Centro de integración
3 "POINT (-75.432705 6.0291789)",02,Creaciones latinas
4 "POINT (-75.3693569 6.150095899999999)",04,Opción Calificada
5 "POINT (-75.26764709999999 6.138652)",06,Sodecon Confecciones
6 "POINT (-75.4295337 6.0350586)",08,Línea modelo
7 "POINT (-75.2622344 6.139473499999999)",13,ALIANZA CONAZZTEX
8 "POINT (-75.337484 6.088041)",19,MANANTIALES
9 "POINT (-75.2598529 6.136884299999999)",24,ECOOELSA
```

Salidas: "1.1-coordenadas.csv" y "1.2-avoid.csv"

Archivos .csv en formato:

```
1.1-coordenadas.csv
1 Nombre,Descripción,lon,lat
2 CI,Centro de integración,-75.57969539999999,6.2171745
3 02,Creaciones latinas,-75.432705,6.0291789
4 04,Opción Calificada,-75.3693569,6.150095899999999
5 06,Sodecon Confecciones,-75.26764709999999,6.138652
6 08,Línea modelo,-75.4295337,6.0350586
7 13,ALIANZA CONAZZTEX,-75.2622344,6.139473499999999
8 19,MANANTIALES,-75.337484,6.088041
9 24,ECOOELSA,-75.2598529,6.136884299999999
```

Cálculo de distancias y tiempos: 2-Distancias.py

A partir de las coordenadas preprocesadas, se calculan las distancias y tiempos de viaje entre todos los pares de puntos mediante el servicio de direcciones de ORS. Aunque ORS ofrece la posibilidad de generar matrices directamente, se optó por el uso del endpoint de direcciones, ya que permite un mayor control sobre el cálculo, incluyendo la definición del perfil vehicular, la restricción de tipos de vías y la incorporación de puntos o zonas a evitar.

Para el uso de ORS se requiere una cuenta y una clave API, con créditos limitados en su versión gratuita. Por este motivo, el cálculo de rutas se realiza una única vez. Para cada par origen – destino se genera un archivo GeoJSON que almacena la geometría de la ruta, así como su distancia y duración, permitiendo reutilizar esta información sin necesidad de recalculer el direccionamiento, estos archivos se guardan en una carpeta nombrada 'rutas_geojson'.

El servicio de direcciones de ORS requiere que las coordenadas se expresen en el orden longitud, latitud. Adicionalmente, los puntos avoid deben definirse como polígonos o multipolígonos, por lo que se emplea una función auxiliar que transforma las coordenadas de restricción en el formato requerido. El cálculo se realiza utilizando el perfil vehicular HGV y la ruta recomendada por el servicio.

Entradas: "1.1-coordenadas.csv" y "1.2-avoid.csv"

Salidas: "2.1-distancias.csv", "2.2-tiempos.csv", "2.3-Rutas.html", carpeta "rutas_geojson"

- Matriz de distancias en kilómetros.
- Matriz de duraciones en minutos.
- 1056 archivos GeoJSON.
- Mapa con rutas.

Para mayor detalle sobre las opciones del servicio de direcciones de ors, véase:

<https://gisscience.github.io/openrouteservice/api-reference/endpoints/directions/routing-options>

Construcción de la instancia VRP: 3-ProblemData.py

Una vez obtenidas las matrices de distancia y tiempo, se procede a la construcción de la instancia del problema de ruteo de vehículos mediante la librería PyVRP. Este proceso se realiza de forma secuencial y modular, abarcando la lectura de datos, el escalamiento numérico y la definición de los distintos componentes del modelo: depósitos, flota, clientes y arcos.

Lectura e integración de datos de entrada

Entradas: "1.1-coordenadas.csv", "2.1-distancias.csv", "2.2-tiempos.csv", "demanda.csv", "service.csv"

Salidas: "3.1-ProblemData.pkl", "3.2-Modelo.pkl"

El script inicia con la lectura de los archivos necesarios para la construcción del modelo:

- Archivo de coordenadas, que contiene la ubicación geográfica de depósitos y plantas.
- Matriz de distancias, expresada en kilómetros.
- Matriz de tiempos de viaje, expresada en minutos.
- Archivo de demandas, que especifica las cantidades a entregar y recoger por planta.
- Archivo de tiempos de servicio asociados a cada operación.

Estos datos se integran mediante estructuras tabulares, permitiendo asociar cada planta con su ubicación, demanda y tiempo de atención correspondiente.

```
coords = pd.read_csv("1.1-coordenadas.csv")
dist = pd.read_csv("2.1-distancias.csv", header=None, index_col=False)
time = pd.read_csv("2.2-tiempos.csv", header=None, index_col=False)
dem = pd.read_csv("demanda.csv")
service = pd.read_csv("service.csv")
```

Escalamiento numérico

PyVRP requiere que todas las magnitudes relacionadas con capacidades, demandas y tiempos sean números enteros. Para cumplir con esta restricción, se aplica un factor de escala uniforme de 100 a las matrices de distancia y tiempo, así como a las demandas y tiempos de servicio. Este escalamiento preserva las proporciones originales y permite una representación precisa durante el proceso de optimización.

```
# Modelo
m = Model()
locations = {}
#-----
# Escalar
SCALE = 100
dist = round(dist*SCALE)
time = round(time*SCALE)
```

Definición de los depósitos

Se definen dos depósitos en el modelo:

- Centro de Integración (CI), como depósito de inicio de las rutas.
- Centro de Distribución (CD), como depósito de finalización.

Cada depósito se agrega al modelo mediante el método `add_depot`, especificando sus coordenadas geográficas y un identificador único. Esta configuración permite modelar rutas abiertas, donde el origen y el destino de los vehículos no coinciden.

```
# ===== Depots =====  
# Busca los nombres de "CI" y "CD" en el .csv de coordenadas  
CI = coords[coords["Nombre"] == "CI"].iloc[0]  
CD = coords[coords["Nombre"] == "CD"].iloc[0]  
# Agregar al modelo como depósito  
locations["CI"] = m.add_depot(  
    x=float(CI["lon"]),  
    y=float(CI["lat"]),  
    name=CI["Nombre"]  
)  
locations["CD"] = m.add_depot(  
    x=float(CD["lon"]),  
    y=float(CD["lat"]),  
    name=CD["Nombre"]  
)
```

Definición de la flota vehicular

La flota se configura como heterogénea, asignando a cada tipo de vehículo una capacidad específica. Para cada vehículo se definen los siguientes parámetros:

- Capacidad máxima de carga.
- Número de vehículos disponibles por tipo.
- Depósito de inicio y de finalización.
- Duración máxima del turno de operación.
- Límite de sobretiempo permitido y su costo asociado.

- Costos unitarios de distancia y tiempo.

Esta configuración permite reflejar condiciones operativas reales, incluyendo restricciones de jornada laboral y penalizaciones por excedentes de tiempo.

```
# ===== Vehicles =====
# Capacidades
VEH_CAPS = {
    "STE138": 31,
    "WCP677": 22,
    "WCP384": 20,
    "PUN354": 15,
    "JY0449": 24,
}
# Agregar al modelo como vehículos
for name, cap in VEH_CAPS.items():
    m.add_vehicle_type(
        capacity=[cap*0.9*SCALE],
        num_available=1,
        start_depot=locations["CI"],
        end_depot=locations["CD"],
        shift_duration=10*60*SCALE,
        unit_distance_cost=1,
        unit_duration_cost=1,
        # reload_depots=[locations["CI"]],
        max_overtime=3*60*SCALE,
        unit_overtime_cost=100000,
        name=name,
    )
```

Modelado de clientes

Cada planta se modela como uno o más clientes en función de sus operaciones logísticas. Se distingue explícitamente entre entregas y recogidas, creando nodos independientes para cada tipo de operación. Esta separación garantiza que ambas actividades no se realicen estrictamente de manera simultánea en el modelo.

Para las entregas, se aplica una estrategia de split delivery únicamente cuando la cantidad a entregar supera el 70% de la capacidad máxima del vehículo. En estos casos, la demanda se

divide en múltiples subnodos mediante una estrategia Fully-Flexible Two-Splitting (FF2S)

(Becker & Schneider, 2025). Solo el último subnodo asociado a una entrega requiere tiempo de servicio, mientras que los subnodos intermedios se consideran únicamente como operaciones de carga.

```
# --- Entregas ---
# Si la demanda de una planta ocupa más del 70% de la capacidad del vehículo, se divide esta carga
if delivery > 0:
    MAX_CAP = max(VEH_CAPS.values()) * SCALE
    if delivery > 0.7 * MAX_CAP:
        parts = split_FF2S(delivery)
    else:
        parts = [delivery]
    for k, part in enumerate(parts, start=1):
        # Nombrar los puntos
        name_d = f"{planta}d_{k}" if len(parts) > 1 else f"{planta}d"
        # Ventanas de tiempo
        if planta in ("A6", "42"):
            tw_e = 0
            tw_l = 90 * SCALE
        else:
            tw_e = 0
            tw_l = 1440 * SCALE
        # tiempo servicio SOLO en el último subnodo
        service_k = srv_di if k == len(parts) else 0
        # Agregar al modelo como clientes
        locations[name_d] = m.add_client(
            x=float(row["lon"]),
            y=float(row["lat"]),
            delivery=[part],
            pickup=[],
            service_duration=service_k,
            tw_early=tw_e,
            tw_late=tw_l,
            required=True,
            name=name_d,
        )
```

```
# --- Recogidas ---
if pickup > 0:
    # Nombrar los puntos
    name_p = f"{planta}p"
    # Ventanas de tiempo
    tw_e = 0
    tw_l = 1440 * SCALE
    # Agregar al modelo como clientes
    locations[name_p] = m.add_client(
        x=float(row["lon"]),
        y=float(row["lat"]),
        delivery=[],
        pickup=[pickup],
        service_duration=srv_pi,
        tw_early=tw_e,
        tw_late=tw_l,
        required=True,
        name=name_p,
    )
```

Cada cliente se agrega al modelo mediante el método `add_client`, definiendo los siguientes atributos:

- Coordenadas geográficas.
- Cantidades de entrega y/o recogida.
- Tiempo de servicio.
- Ventana de tiempo de atención.
- Indicador de obligatoriedad de visita.

Para las plantas A6 y 42 se establece una ventana de tiempo restringida a los primeros 90 minutos de operación, mientras que el resto de los clientes pueden ser atendidos en cualquier momento del día.

```
# Ventanas de tiempo
if planta in ("A6", "42"):
    tw_e = 0
    tw_l = 90 * SCALE
else:
    tw_e = 0
    tw_l = 1440 * SCALE
```

Definición de arcos

Una vez definidos depósitos y clientes, se construyen los arcos del modelo, que representan los posibles desplazamientos entre todas las ubicaciones. Para cada par de nodos se asignan los valores de distancia y duración obtenidos previamente a partir de los archivos GeoJSON generados por ORS.

Dado que el modelo puede contener múltiples nodos asociados a un mismo punto físico (debido al split delivery), se utiliza una correspondencia entre nodos derivados y su cliente original para garantizar que los arcos reflejen correctamente las distancias y tiempos reales. No se consideran arcos triviales entre nodos que representan el mismo punto físico.

Para una descripción detallada de las capacidades y parámetros del modelo, véase la documentación oficial de PyVRP: <https://pyvrp.org/api/pyvrp.html>

Almacenamiento de la instancia

Finalmente, el modelo construido puede serializarse y almacenarse en disco para su posterior resolución o análisis, permitiendo separar el proceso de construcción de la instancia del proceso de optimización.

```
# Guardar instancia
# Exportar en .pkl
problem = m.data()
with open("3.1.1-ProblemData.pkl", "wb") as f:
    pickle.dump(problem, f)
with open("3.2.1-Modelo.pkl", "wb") as f:
    pickle.dump(m, f)
```

Resolución de modelo: 4-Solve.py

Entradas: "3.2-Modelo.pkl"

Salidas: "4.1-Res.pkl", "4.2-Rutas.html"

Una vez construida la instancia del problema, se procede a su resolución mediante la librería PyVRP, así como a la interpretación y visualización de los resultados obtenidos. Este script se encarga de ejecutar el algoritmo de optimización, extraer la solución factible encontrada y transformarla en información operativa comprensible.

Carga de la instancia del modelo

El proceso inicia con la carga del modelo previamente construido y almacenado en disco. Esta separación entre la construcción de la instancia y su resolución permite reutilizar el mismo modelo para distintos escenarios de optimización sin necesidad de reconstruir la estructura completa del problema.

```
# Modelo
with open('3.2-Modelo.pkl', 'rb') as f:
    m = pickle.load(f)
```

Configuración del proceso de resolución

La resolución del VRP se realiza con *pyvrp.solve*, empleando como criterio de parada un tiempo máximo de ejecución (en segundos), tras el cual el algoritmo retorna la mejor solución encontrada.

Esta configuración permite balancear la calidad de la solución con el tiempo computacional, aspecto relevante en contextos operativos donde se requieren resultados en tiempos acotados.

```
# --- Resolver ---  
res = m.solve(MaxRuntime(600))  
solution = res.best
```

Extracción e interpretación de la solución

La resolución del problema se realiza utilizando la librería PyVRP, la cual retorna la solución en forma de un conjunto de rutas, cada una asociada a un vehículo, con una secuencia ordenada de visitas y métricas operativas como distancia, duración, cargas y factibilidad.

Dado que la solución entregada por PyVRP se expresa a nivel de nodos del modelo, los resultados se presentan inicialmente en consola de manera estructurada, indicando para cada ruta el vehículo asignado, el depósito de inicio y finalización, el orden de visitas y los valores agregados de distancia, tiempo, entregas y recogidas.

Con el fin de facilitar la interpretación operativa de la solución, se genera adicionalmente un mapa interactivo empleando Folium, en el cual se integran los principales elementos de interés: ubicación de depósitos y clientes, rutas asignadas a cada vehículo, orden de atención, información detallada por tramo (distancia, duración y nivel de ocupación del vehículo) y un resumen consolidado por ruta. Esta visualización permite analizar de forma intuitiva la factibilidad y estructura de las rutas obtenidas.

Modificación y extensión del modelo

La implementación permite realizar modificaciones y extensiones de manera localizada sin alterar la estructura general del modelo. Es importante resaltar que el proceso está

estructurado como una cadena de scripts consecutivos; por lo tanto, todo archivo cuyo nombre sea modificado o reemplazado debe actualizarse explícitamente en los scripts donde sea utilizado, ya que de lo contrario se interrumpirá el flujo de construcción y solución de la instancia. A continuación, se describen los principales puntos de intervención para adaptar el modelo a nuevos escenarios operativos.

- **¿Cómo agregar un nuevo cliente?**

Para incorporar un nuevo cliente al modelo, o modificar alguno ya existente, el procedimiento es el siguiente:

- Opción 1: e.g. cambiar la ubicación de una planta, o agregar una con coordenadas conocidas previamente.
 - Incluir manualmente la información del cliente en todos los .csv (No ejecutar 1-Preprocesado.py)
 - Recalcular las distancias y tiempos (*2-Distancias.py*)
 - Reconstruir la instancia (*3-ProblemData.py*)
 - Resolver (*4-Solve.py*)
- Opción 2: e.g. si son varias plantas para agregar; crear una nueva instancia.
 - Establecer en un mapa las ubicaciones de las plantas, y exportar el archivo .csv.
 - Actualizar el nombre del archivo en *1-Preprocesado.py*. Si cambia el nombre del segundo argumento, actualizar (reemplazar en la línea 12 de *2-Distancias.py*).
 - Recalcular las distancias y tiempos (*2-Distancias.py*).
 - Reconstruir la instancia (*3-ProblemData.py*).

- Resolver (*4-Solve.py*).

- **¿Cómo agregar un nuevo vehículo?**

Para incorporar un nuevo vehículo al modelo, o modificar alguno ya existente, el procedimiento es el siguiente:

- Buscar en *3-ProblemData.py* el apartado de 'Vehicles'.
- Actualizar el diccionario VEH_CAPS, que está de la forma {"nombre del vehículo": capacidad}
- Ajustar los otros parámetros de ser necesario.
- e.g. voy a agregar dos vehículos al modelo actual, pero no tiene las mismas configuraciones para la jornada (shift_duration), podría configurarse así: agregar los vehículos en el diccionario de VEH_CAPS, definir una variable con la jornada estándar "jornada", y la jornada diferente de los nuevos "jor_nuevos", y, usar estos valores en los parámetros al agregarlos en el modelo.


```
# ===== Vehicles =====
# Capacidades
VEH_CAPS = {
    "STE138": 31,
    "WCP677": 22,
    "WCP384": 20,
    "PUN354": 15,
    "JYO449": 24,
    # Nuevos vehículos
    "NUEVO_1": 28,
    "NUEVO_2": 18,
}

# Jornada estándar y jornada alternativa
jornada = 10 * 60 * SCALE
jor_nuevos = 8 * 60 * SCALE # nueva jornada común para los vehículos agregados

# Identificación de vehículos con jornada alternativa
nuevos = {"NUEVO_1", "NUEVO_2"}

# Agregar vehículos al modelo
for name, cap in VEH_CAPS.items():
    m.add_vehicle_type(
        capacity=[cap * 0.9 * SCALE],
        num_available=1,
        start_depot=locations["CI"],
        end_depot=locations["CD"],
        shift_duration=(
            jor_nuevos if name in nuevos else jornada
        ),
        unit_distance_cost=1,
        unit_duration_cost=1,
        max_overtime=3 * 60 * SCALE,
        unit_overtime_cost=100000,
        name=name,
    )
```

- **¿Cómo agregar una ventana de tiempo?**

Las ventanas de tiempo definidas en el modelo no representan horarios absolutos de reloj, sino intervalos expresados en función del tiempo transcurrido desde el inicio de la jornada de los vehículos. En la implementación actual, se asume que la operación comienza a las 05:00 a.m., por lo que el tiempo cero del modelo corresponde a ese instante. En consecuencia, los valores de `tw_early` y `tw_late` deben interpretarse como tiempos relativos al inicio de la ruta, e incluyen de manera implícita el tiempo de desplazamiento desde el depósito hasta cada planta. Esto implica que la definición de una ventana de atención no puede trasladarse de forma directa desde el horario operativo real, sino que requiere un ajuste que considere las distancias, velocidades y condiciones de

recorrido modeladas. Por esta razón, la asignación de ventanas de tiempo se realiza de manera iterativa y aproximada, “jugando” con los límites inferior y superior del intervalo, de modo que reflejen de forma razonable el momento en el cual un vehículo puede llegar a la planta dentro de la jornada.

Para incorporar un nuevo rango de tiempo de atención de un cliente al modelo, o modificar alguno ya existente, el procedimiento es el siguiente:

- Buscar en *3-ProblemData.py* el apartado de ‘Clientes’.
- ¿La ventana temporal es para recoger, o entregar? Buscar ‘entregas’ o ‘recogidas’ según corresponda.
- Buscar el comentario ‘# Ventanas de tiempo’ y, en el ciclo if cambiar o agregar en plantas (‘planta1’, ‘planta2’...) y los argumentos *tw_e* (hora más temprana) y *tw_l* (hora más tarde), estos en minutos por la escala establecida.
- e.g., voy a ponerle una ventana temporal a las plantas 49 y A5 para entregar entre los minutos 90 y 120 de recorrido.

```
# --- Entregas ---
if delivery > 0:
    MAX_CAP = max(VEH_CAPS.values()) * SCALE
    if delivery > 0.7 * MAX_CAP:
        parts = split_FF2S(delivery)
    else:
        parts = [delivery]
    for k, part in enumerate(parts, start=1):
        name_d = f"{planta}d_{k}" if len(parts) > 1 else f"{planta}d"
        # Ventanas de tiempo (existentes)
        if planta in ("A6", "42"):
            tw_e = 0
            tw_l = 90 * SCALE
        # Ventanas de tiempo (nuevas)
        elif planta in ("13", "46"):
            tw_e = 90 * SCALE
            tw_l = 120 * SCALE
        else:
            tw_e = 0
            tw_l = 1440 * SCALE
        service_k = srv_di if k == len(parts) else 0
        locations[name_d] = m.add_client(
            x=float(row["lon"]),
            y=float(row["lat"]),
            delivery=[part],
            pickup=[],
            service_duration=service_k,
            tw_early=tw_e,
            tw_late=tw_l,
            required=True,
            name=name_d,
        )
    )
```

Limitaciones de la implementación

Si bien la implementación desarrollada permite modelar y resolver el problema de ruteo bajo condiciones operativas realistas, existen algunas limitaciones inherentes a las decisiones de modelado y a las herramientas empleadas. En primer lugar, el cálculo de distancias y tiempos depende del servicio de ORS, lo que implica una dependencia de disponibilidad, límites de uso y posibles variaciones en los resultados ante cambios en la red vial subyacente.

Adicionalmente, el uso de un factor de escala para representar demandas, capacidades y tiempos, aunque necesario por las restricciones de PyVRP, introduce una aproximación discreta que puede generar pequeñas desviaciones respecto a los valores reales.

La estrategia de split delivery aplicada se limita a un umbral fijo y a una heurística específica, lo que restringe la exploración de otras políticas de fraccionamiento de carga que podrían resultar más eficientes en determinados escenarios. Asimismo, las ventanas de tiempo consideradas son relativamente rígidas y no contemplan incertidumbre en los tiempos de viaje o de servicio.

Finalmente, aunque la visualización permite un análisis detallado de las rutas obtenidas, esta se realiza como un proceso posterior a la optimización, sin retroalimentación directa al modelo durante la resolución.

Reproducibilidad

La implementación fue desarrollada siguiendo un enfoque modular, separando el preprocesamiento de datos, la construcción de la instancia, la resolución del modelo y la visualización de resultados. Esta estructura permite reproducir los experimentos y evaluar escenarios alternativos de manera controlada.

Todos los scripts, junto con los archivos de entrada necesarios, se encuentran disponibles en un repositorio público. El uso de archivos GeoJSON para almacenar las rutas calculadas permite replicar los resultados sin necesidad de recalcular distancias y tiempos, reduciendo la dependencia directa del servicio de direccionamiento externo.

La serialización del modelo y de los resultados facilita la separación entre las etapas de construcción y resolución, permitiendo ejecutar nuevamente la optimización bajo distintos criterios de parada o analizar soluciones previamente obtenidas. En conjunto, estas decisiones garantizan la trazabilidad y reproducibilidad del proceso computacional descrito.

Bibliografía

Becker, C., & Schneider, M. (2025). An A-Priori-Splitting-Based Heuristic for the Split Delivery

Vehicle Routing Problem With Time Windows. *Networks*, 86(4), 468–516.

<https://doi.org/10.1002/net.70004>

Wouda, N.A., L. Lan, and W. Kool (2024). PyVRP: a high-performance VRP solver package.

INFORMS Journal on Computing, 36(4): 943-955.

<https://doi.org/10.1287/ijoc.2023.0055>